

Designed by Remus Teodorescu 2026

LECTURE 1.2 · RECORDED

NumPy, and the Shape of Everything

Remus Teodorescu

With support from Research Assistant Noman Khan - nomank@energy.aau.dk

Aalborg University · ret@et.aau.dk

Deep Learning for Engineering · 2026

THE RULE, IN ONE SENTENCE

Aligning the shapes from the right, every pair of lengths must be equal, or one of them must be 1.

(6,240)	6	240
(6,1)	6	1
result	**6**	**240**

There is no second clause. Everything in this lecture follows from that one sentence: what an array is, why a slice shares memory with what it came from, why two shapes combine when you expected an error, and how to draw a figure another engineer will believe. It is the rule behind the most expensive mistake in this course.

Learning Objectives

SECTIONS, IN ORDER

1. **The array** — the object, its dtype and shape, indexing and views, and why it is fast

2. **Broadcasting** — the rule, the silent failure, reshaping and reductions

3. **Figures** — a figure an engineer can defend, several curves, log axes, the residual plot

One habit runs through all three, and the exercises enforce it: print the shape of every array on the line after you make it.

READ, AND WHAT EACH ANSWERS

NumPy basics	what is an array, and why not a list?
NumPy broadcast	why did those two shapes combine?
Matplotlib	would another engineer believe this figure?

Array

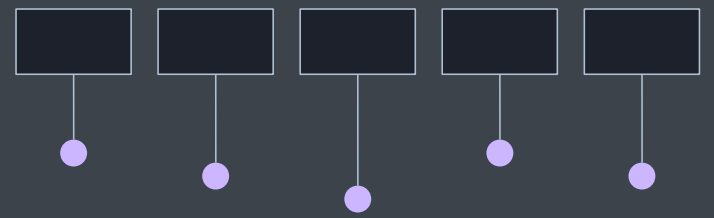
Array — a single contiguous block of memory holding elements that all have the same type, with a shape that says how to read it. A list holds references to objects that may live anywhere and be of any type. That one difference is where the speed, the elementwise arithmetic and the shape rules all come from.

ONE BLOCK OF MEMORY, ONE TYPE, ONE SHAPE

Arithmetic on an array is elementwise and happens in compiled code. Multiplying a list by two repeats the list; multiplying an array by two doubles every element.

The memory is contiguous, so the machine walks it without chasing pointers, and the type is fixed, so an array of integers stays an array of integers whatever you assign into it.

A LIST — REFERENCES, ANYWHERE



five pointers, five objects, five types allowed

AN ARRAY — ONE BLOCK, ONE TYPE



contiguous float64, shape (5,)

```
np.array([1.0, 2.0, 3.0, 4.0, 5.0]) * 2
doubles every element
```

dtype

dtype — the type of an array's elements, fixed when the array is made and enforced afterwards. Whole numbers give int64 and anything with a decimal point gives float64. It is enforced silently, which is what makes it worth a slide: assigning a float into an integer array truncates it and says nothing.

FIXED WHEN THE ARRAY IS MADE, ENFORCED AFTER

Two point seven becomes two, with no warning, and the array is an integer array still. If a result is mysteriously blocky or zero, print the dtype before anything else.

The second trap matters in the PyTorch lecture. NumPy defaults to float64 and PyTorch to float32, so an array handed over without a dtype argument produces a warning at best and a silent conversion at worst.

THE FOUR TYPES IN THIS COURSE

float64	NumPy's default — use it here
float32	torch's default — L2.2 onwards
int64	counts and indices only
bool	masks, from slide 7

```
counts = np.array([1, 2, 3])
counts.dtype                int64
counts[0] = 2.7             # truncates
counts[0]                   2

x = np.array([1, 2, 3], dtype=np.float64)
```

Say the dtype when it matters. It matters at every torch boundary.

Shape

Shape — a tuple giving the length of each axis. Its length is the number of dimensions and the product of its entries is the number of elements. Read it right to left as a nesting: six comma two hundred and forty is six rows of two hundred and forty, so axis zero indexes the rows.

PRINT IT. NOT SOMETIMES — EVERY TIME.

The habit this slide exists to install: every time you build an array, reshape one, or get one back from a function, print its shape on the very next line.

It sounds like advice nobody follows. The exercise notebooks make you do it anyway, and by the fourth exercise most people are doing it without being asked.

TALK · 2 MIN What does a shape tuple tell you that the numbers in the array do not? Then, in the grid on the right, which axis would you average over to get one number per gauge, and what shape comes back?

ONE ARRAY, READ CORRECTLY



shape (6, 240) — drawn small

axis 0 → gauges

axis 1 → samples

THE FOUR CONSTRUCTORS

```
np.linspace(0.0, 2.0, 41)   # endpoint included
np.arange(0, 6)             # endpoint excluded
np.zeros((2, 5))            # a shape tuple
np.random.normal(size=(3, 4))
```

Indexing and Views

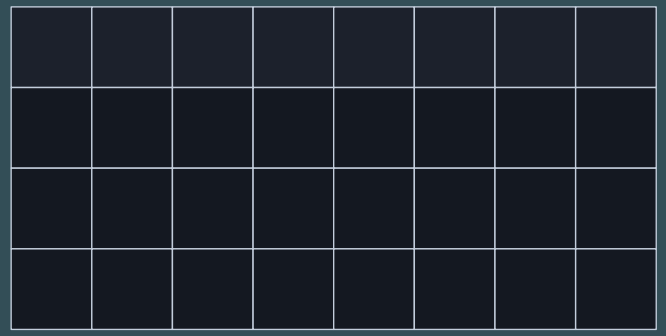
Indexing — reaching into an array with one index per axis, inside a single pair of brackets. The sentence to remember is short: indexing with an integer removes an axis, and slicing with a colon keeps it. A slice is also a view, which means it shares memory with the array it came from.

AND THE DIFFERENCE BEHIND THE SILENT BROADCAST

So the first row of a six by two hundred and forty array, taken with an integer, has one axis. Taken as a slice of length one, it has two.

Those two objects hold identical values and behave differently the moment you combine them with anything else. That difference is the seed of the failure two slides from now.

THE SAME NUMBERS, TWO SHAPES



```
a[0, :]   →  shape (8,)
a[0:1, :] →  shape (1, 8)
```

```
a[1, 2]      one element, shape ()
a[:, 0]      first column, shape (4,)
a[0, :]      first row, shape (8,)
a[0:1, :]    first row, shape (1, 8)
a[1][2]      works, and do not write it
```

An integer removes an axis. A colon keeps it. Remember that one.

Vectorisation

Vectorisation — writing an expression over whole arrays instead of looping over their elements, so that the arithmetic happens once in compiled code rather than a million times in the interpreter. It is the reason NumPy exists, and the speed-up is about two orders of magnitude.

THE SAME FORMULA THREE WAYS, 200,000 POINTS

On a typical Colab processor the loop takes about a tenth of a second, the comprehension a little less, and the array expression about a millisecond.

The comprehension is barely faster than the loop, because the arithmetic still happens one Python object at a time. Writing a loop more compactly does not make it array code.

$$y(x) = \frac{w\,x^2(6L^2 - 4Lx + x^2)}{24\,E\,I}$$

w the load per length
L the span, EI the stiffness

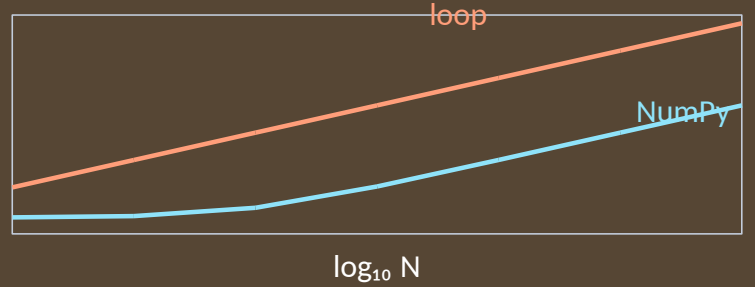
THE WHOLE CALCULATION, VECTORISED

```
k = 6*L**2 - 4*L*x + x**2        # x is 200 000 positions at once
y = w * x**2 * k / (24*E*I)      # one expression, no loop
```

200,000 POINTS, BEST OF FIVE RUNS

for loop	≈ 100 ms
comprehension	≈ 90 ms — barely better
NumPy	≈ 1 ms
speed-up	tens to low hundreds

AND IT WIDENS WITH SIZE



Schematic — axes are orders of magnitude, and deliberately not calibrated.

Broadcasting

Broadcasting — two shapes are compatible if, aligning them from the right, every pair of lengths is either equal or one of them is one. Missing axes on the left of the shorter shape count as one, and an axis of length one is stretched to match.

THERE IS NO SECOND CLAUSE

Aligning from the right is the half people forget. Shapes are compared by their last axis first, so a one-dimensional array lines up with the last axis of whatever it meets, whatever you intended.

Write the two shapes out, one above the other, right-aligned, and check the columns. That is what the table on the right does, and what the broadcast_table function in the exercise module prints for you.

THE PROCEDURE, AS A PICTURE

(6,240)	6	240
(240,)	·	240
result	6	240

240 against 240 — equal. Then 6 against a missing axis, which counts as 1 and is stretched. The one-dimensional array is applied to every row.

Equal, or one. Aligned from the right.

When It Goes Wrong

The silent broadcast — two arrays whose shapes differ in a way the rule happily resolves, producing an answer of the wrong shape and no error at all. One hundred predictions against one hundred measurements shaped one hundred by one gives a hundred by a hundred, and the mean of it is a plausible wrong number.

THE MOST EXPENSIVE MISUNDERSTANDING IN THIS COURSE

Nothing raises. You take the mean, square-root it, and write down an error that is wrong by a factor of a few, always plausibly.

Two defences. Print both shapes on the line above the subtraction, and when a number surprises you check the shape of what produced it before you question the physics.

NO ERROR, AND THE WRONG SHAPE

(100,)	·	100
(100,1)	100	1
result	**100**	**100**

Both stretched. 10,000 elements, silently.

THE ONE-SECOND DEFENCE

```
print(pred.shape, meas.shape)
(100,) (100, 1)

err = pred - meas.ravel()
err.shape                  (100,)
```

Adding an Axis

Reshaping — changing how the same block of memory is read. None inside the brackets inserts an axis of length one, reshape states the shape you want, ravel flattens to one dimension, and squeeze removes every axis of length one.

FOUR TOOLS, AND WHEN EACH IS THE RIGHT ONE

Reach for None when you are setting up a broadcast on purpose: it is short, and it says that is what you meant.

Squeeze is convenient and slightly dangerous, because it removes axes you did not think about as well as the one you did.

THE FOUR, AND WHAT THEY SAY

x[:, None]	add an axis, for a broadcast
x.reshape(6, 1)	state the shape you want
x.ravel()	flatten to (n,)
x.squeeze()	drop every length-1 axis

```
x.shape            (6,)
x[:, None].shape   (6, 1)   a column
x[None, :].shape   (1, 6)   a row
x.reshape(-1, 1)   (6, 1)   same thing
```

A column is (n, 1). L2.2 makes that the convention for every network input.

Reductions

Reduction — an operation that collapses an axis: sum, mean, max, and the rest. The axis argument names the axis that disappears, which is the opposite of how most people first read it, and leaving it out collapses everything to a single number.

THE AXIS ARGUMENT NAMES WHAT DISAPPEARS

So for an array of six gauges by two hundred and forty samples, mean over axis one gives six numbers, one per gauge, and mean over axis zero gives two hundred and forty.

Say the sentence to yourself the first few times: the axis in the argument is the one that goes away.

WHICH AXIS DISAPPEARS




```
a.shape                          (6, 240)
a.sum(axis=0).shape              (240,)
a.sum(axis=1).shape              (6,)
a.mean(axis=1, keepdims=True)    (6, 1)

centred = a - a.mean(axis=1, keepdims=True)
```

That last line is why keepdims exists.

A Defensible Figure

A defensible figure — one that another engineer can read without asking you anything: axes labelled with quantities and units, a legend when there is more than one curve, and a scale chosen so the thing being shown is visible. It is not decoration, it is the report.

ONE RULE, AND IT IS NOT NEGOTIABLE

Every figure in every exercise of this course is marked on this. The physics can be right and the figure can still fail to show it.

The rule is one line long: no axis without a unit. It is the commonest single deduction in the reports, and the cheapest to avoid.

THE SKELETON

```
fig, ax = plt.subplots(figsize=(7.2, 4.4))
ax.plot(x_m, y_mm, label='w = 500 N/m')
ax.set_xlabel('position x  [m]')
ax.set_ylabel('deflection  [mm]')
ax.legend()
ax.grid(True, alpha=0.3)
fig.tight_layout()
```

THE CHECKLIST

Both axes labelled. Both units stated. A legend if there is more than one curve. A grid you can read a value off. And a caption that says what the figure shows rather than repeating its title.

Several Curves

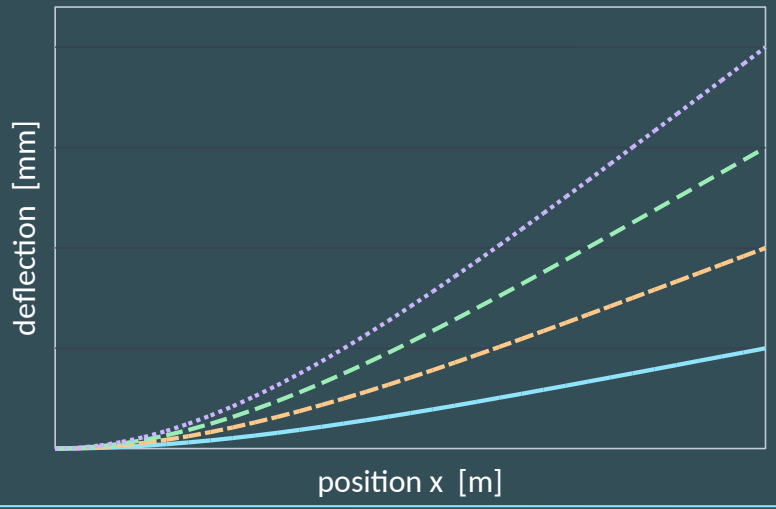
Several curves — more than one line on one pair of axes, which needs them told apart without the reader guessing: a legend, distinct colours, and a line style that survives being printed in black and white or seen by somebody who cannot separate red from green.

THREE HABITS THAT COST NOTHING

Label the curves in the call that draws them rather than afterwards, so a curve and its name cannot drift apart when the order changes.

If the curves cross, say in the caption which is which at the left-hand edge. A legend in the corner is not always enough.

A LOAD SWEEP, DONE PROPERLY



125 N/m	solid
250 N/m	dashed
375 N/m	long dash
500 N/m	dotted

Logarithmic Axes

Logarithmic axes — a scale on which equal distances are equal ratios, which is the right one whenever the quantity spans decades or the relationship is a power law. On a linear axis a decade of variation at the bottom of the range is invisible, and a straight line on a log-log plot is a power law whose exponent is its slope.

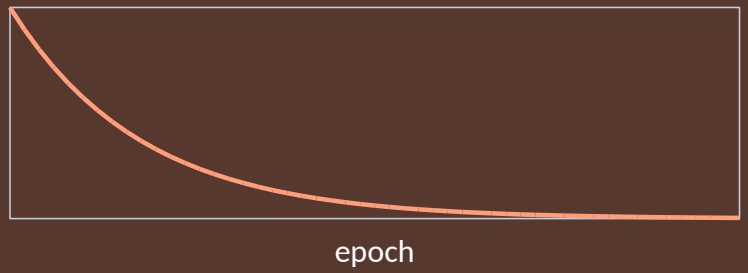
THE COMMONEST AXIS CHANGE

A residual falling from ten to the minus one to ten to the minus six is four flat lines and one drop on a linear axis, and a straight line on a logarithmic one.

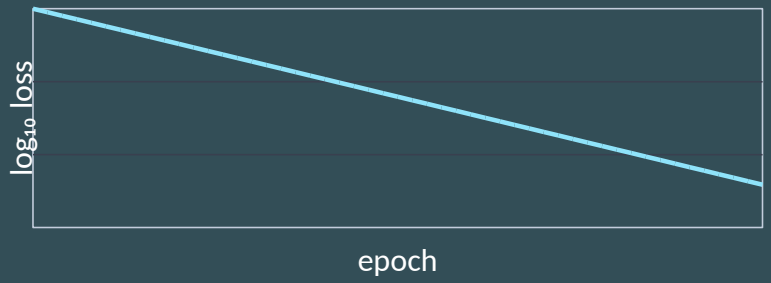
Every training curve in this course is drawn this way, which is why a loss plot with a linear vertical axis usually means somebody has not looked at it.

TALK · 2 MIN What is a logarithmic axis actually showing you that a linear one is not? Then, in the plot on the right, at which point does the linear version stop telling you anything, and what would you have concluded from it?

LINEAR — LOOKS FINISHED AT 200



LOG — STILL FALLING AT 1000



Matplotlib user guide

The Residual Plot

The residual plot — the difference between a model and the measurements, plotted against the input rather than summarised as one number. A single error figure says how big the disagreement is; the residual plot says whether the model is wrong in a pattern, which is the question that matters.

THE FIGURE THAT SHOWS A MISSING TERM

Structure in a residual plot means the model is missing a term. Noise with no structure means it is not.

This is the plot that separates a fit that is good enough from one that happens to have a small error, and it costs one extra line to draw.

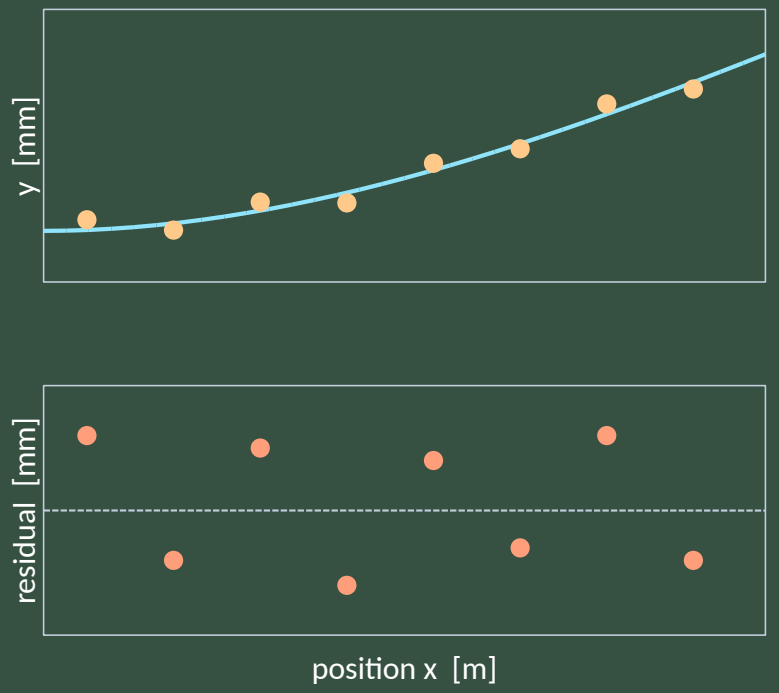
$$\mathrm{RMSE} = \sqrt{\frac{1}{N}\sum_{i=0}^{N-1}\left(\hat{y}_i - y_i\right)^2}$$

$\hat{y}_i$ the model, y_i the reading
N the number of points

THE RESIDUAL, AND ITS RMS

```
r = y_meas - y_model            # both (N,), never (N,1)
rms = np.sqrt((r**2).mean())    # one number, after the plot
```

TWO PANELS, ONE X-AXIS



No pattern in the lower panel. That is what a good fit looks like.

Advanced Topics

RELATED TOPICS LEFT OUT, AND WHERE THEY MATTER

topic	what it adds	read	where it makes a difference
structured arrays	named columns of mixed type	NumPy user guide	logger files, before pandas
fancy indexing	index with an array of positions	NumPy basics, indexing	selecting events, sorting
masked arrays	missing values carried explicitly	NumPy user guide	gappy measurements
einsum	one notation for every contraction	NumPy reference	tensor algebra, attention
random generators	seeds you can actually reproduce	NumPy random guide	any result you report

WHY THEY ARE LEFT OUT

Each is a refinement of the array you now have, and none is needed to write the exercises. Two of them return later: the random generator in every experiment you report, and einsum inside the attention lecture.

EXERCISE

Exercise 1

NOTEBOOK 02 IS THE ONE TO PROTECT

Notebook 02 is the longest in Ex01 and it says so in its first paragraph. Seven TODO cells: arrays against lists, dtype and shape, indexing, then four separate exercises on broadcasting, then the timed comparison.

Three of those broadcasting exercises deserve naming. One asks you to predict eight result shapes before running anything — predict, then check. One makes you produce the shape-one-hundred against shape-one-hundred-by-one bug deliberately and prints both RMS values side by side. And one builds a distance matrix, where the N by N result is what you wanted.

Notebook 03 is figures: the load sweep, the log axis, and the two-panel residual plot. Two TODO cells and a checklist you should keep.

If you are short of time, do notebook 02 properly and skim notebook 03. The figure conventions can be picked up while doing Ex03; the broadcasting cannot be picked up in week seven, which is when you would otherwise need it.

THE NOTEBOOKS

00	environment check — done last time
01	the language — done last time
02	arrays and BROADCASTING — 7 TODOs
03	engineering figures — 2 TODOs

The cantilever of notebook 01 again.

REPORT QUESTION, TO CONCLUDE

Across the notebooks: give one shape mismatch you produced, say what the result should have been and what it actually was, and name the line that would have caught it.

SUMMARY

Key Takeaways

THE ARRAY

what it is	one block of memory, one type, one shape
function	elementwise arithmetic in compiled code
train, tune	print the shape on the line after you make it
vs a list	about a hundred times faster, and typed
used for	every number in the eleven lectures after this

BROADCASTING

what it is	align from the right, stretch any axis of one
function	combines arrays of different shapes, silently
train, tune	write the two shapes out, right-aligned
vs an error	the dangerous case raises nothing at all
used for	every loss and residual you will write

Questions

1 – 2 · THE ARRAY

1 **What is a NumPy array, and where does its speed come from?**

One block of memory with one type and one shape. Arithmetic on it runs element by element in compiled code, with no Python loop, which is about a hundred times faster than a list. Writing the calculation that way is vectorisation. *NumPy basics, arrays; Ex_01 notebook 02*

2 **What does the shape of an array tell you, and what habit goes with it?**

The shape is a tuple with one length per axis: (3, 4) is three rows of four. Indexing with an integer removes an axis; a slice keeps it. The habit is to print the shape on the line after you make an array. *NumPy basics, shape, indexing*

3 – 4 · BROADCASTING

3 **What is the broadcasting rule?**

Write the two shapes right-aligned and compare them axis by axis: two lengths are compatible if they are equal or one of them is one, and an axis of one is stretched. A missing axis on the left counts as one. *NumPy basics, broadcasting*

4 **Why is broadcasting dangerous, and how do you guard against it?**

It raises nothing when two shapes are compatible by accident: a (100,) array minus a (100, 1) array gives (100, 100), ten thousand numbers, and a loss computed from it is wrong without a warning. Write the two shapes out, and check the shape of the result. *NumPy broadcasting; Ex_01 notebook 03*

NEXT

L2.1 — Scientific Python

Arrays are the object. Next time they carry real measurements: pandas for tabular data, a CSV that is genuinely messy, an ODE solved numerically — and a two-parameter curve fit that a network will later struggle to beat.

BEFORE THE NEXT RECORDING

work	Ex01 notebook 02 — ninety minutes
work	Ex01 notebook 03 — thirty minutes
read	the NumPy Broadcasting page
keep	the figure checklist

Notebook 02 section 4 is the part that matters. If you do nothing else before the next recording, predict those eight shapes and find out which ones you got wrong.